UNIVERSITY OF UDINE

DEPARTMENT OF MATHEMATICS, COMPUTER SCIENCE AND PHYSICS

READERSOURCING 2.0: DOCUMENTATION

MICHAEL SOPRANO AND STEFANO MIZZARO

Contents

List of Figures

List of Tables

1 Introduction

The Readersourcing 2.0 ecosystem was developed as part of a research project co-funded by SISSA Medialab[1] and the University of Udine.[2] It was presented by Soprano et al. [5] at the IRCDL 2019 Conference.[3] The paper is also available on Zenodo.[4] The code and related documentation are available on GitHub.[5]

Initially, a recap of its general architecture is presented, followed by a brief description of the role and purpose of each of its components. Specific aspects, such as the technology used, the internal architecture, and the structure of the database, are also discussed. This is achieved through different types of diagrams adhering to the UML standard unless otherwise specified, drawn according to the style rules proposed by Fowler [2]. The current software requirements and operational procedures are recorded alongside the original architectural description.

2 General Architecture

Readersourcing 2.0 is an ecosystem composed of several components. It includes RS_Server [9] (presented in Section 3), which gathers the ratings given by readers, and RS_Rate [8] (presented in Section 5), which acts as a client and enables readers to rate publications. Although all operations can be carried out directly through the web interface provided by RS_Server, RS_PDF [6] (presented in Section 4) annotates PDF files with a rating URL and QR code on behalf of the server-side application. Finally, RS_Py (presented in Section 6) provides a fully functional Python implementation of the RSM and TRM models described by Soprano et al. [5], together with the notebooks used to generate datasets and analyze experiments.

Figure 1 provides an overview of the architecture of Readersourcing 2.0, and in the following, we briefly describe these four components. We summarize the capabilities of the ecosystem in Section 7.

3 RS_Server

RS_Server [9] is the server-side component responsible for collecting and aggregating the ratings given by readers and applying the RSM and TRM models described by Soprano et al. [5] to compute quality scores for readers and publications. An instance of RS_Server is deployed with a compatible RS_PDF executable, which is shipped in the `lib` directory and invoked as a local process when a publication must be annotated. Up to n browsers and

[1] `https://medialab.sissa.it/`
[2] `https://www.uniud.it/`
[3] `https://ircdl2019.isti.cnr.it/`
[4] `https://zenodo.org/record/1446468`
[5] `https://github.com/Miccighel/Readersourcing-2.0`

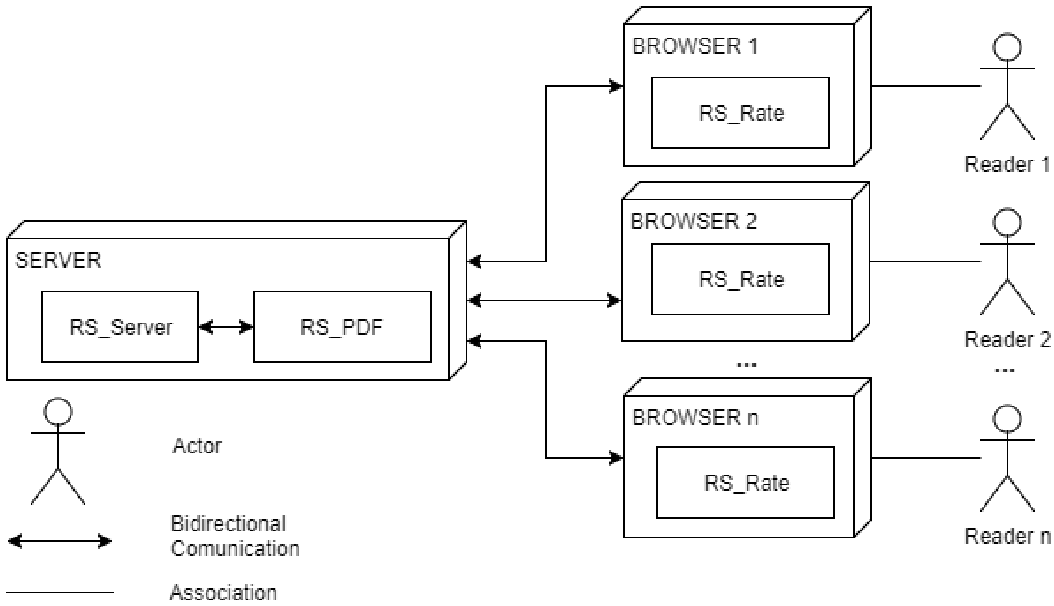


Figure 1: Conceptual architecture of Readersourcing 2.0 (NOT UML). RS_PDF is bundled with and invoked by RS_Server.

their users can communicate with the server, each through an instance of RS_Rate. The extension is distributed for Chromium-based browsers and Firefox.

Readers can interact with the server either through clients installed in their browsers or through the stand-alone web interface provided by RS_Server. These clients manage reader registration and authentication, rating actions, and downloads of link-annotated publications.

During the design phase of RS_Server, strategies were adopted to ensure extensibility and generality. This includes:

(i) Straightforward addition of new models.

(ii) A shared input data format among all models.

(iii) A standard procedure for models needing to save values locally in the RS_Server database.

3.1 Implementation and Technology

RS_Server is developed using *Ruby on Rails*,[6] an open-source web application framework built on the Ruby programming language and strongly based on the Model-View-Controller (MVC) architectural pattern.

[6] `https://rubyonrails.org/`

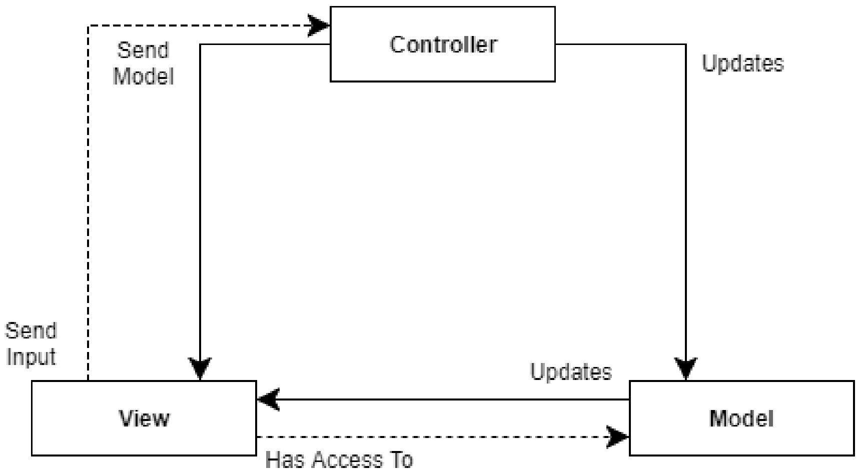


Figure 2: Intuitive scheme of the MVC pattern (NOT UML).

The MVC pattern separates a program's control logic from its data presentation and business logic, helping developers establish a clear architecture from the initial design phase. Figure 2 illustrates the resulting structure. It comprises three distinct entities: the *Controller*, which manages control logic; the *Model*, which encapsulates business logic; and the *View*, which presents data.

The Controller has direct access to both the Model and the View. It typically receives user input from the View, updates the internal state of the Model through its methods, and then passes the updated Model back to the View for presentation. A software system may contain several Controllers, each of which can manage multiple Model instances. In web MVC frameworks such as Rails, Controllers are commonly organized around domain resources, and a Model can be presented through more than one View.

MVC is not the only principle underlying Rails. Another central principle is "Convention Over Configuration": the framework reduces the number of decisions required during application development by providing standard conventions that can still be changed when greater flexibility is needed. The "Rails Doctrine" describes these principles in greater detail.[7]

Rails is an actively developed framework used by organizations such as *GitHub* and *SoundCloud*. It is supported by an established community and extensive learning resources.

3.2 Communication Paradigm

An MVC framework such as Rails can support several kinds of web applications, including a *Web Service*. A Web Service exposes operations remotely by exchanging messages encoded in a standard format such as *JSON* over a transport protocol such as *HTTP*. Its interface

[7] `https://rubyonrails.org/doctrine/`

must define the available resources and operations, together with the messages required to access them.

One communication paradigm for Web Services is *REST* (*REpresentational State Transfer*). In a RESTful interface, resources are identified by URIs and the HTTP method determines the operation to perform. The server returns a response in the agreed interchange format, and the client interprets that response.

RS_Server is a Rails application exposing RESTful interfaces alongside server-rendered web workflows. API clients exchange JSON messages over HTTP, while the same domain and authorization rules are reused by the web interface.

The REST interface is defined by the application routes, JSON views, and integration tests. A Postman collection provides request and response examples for the exposed resources:

`https://documenter.getpostman.com/view/4632696/RWTiwfV4?version=latest`.

For example, Table 1 presents a subset of the RESTful interface of RS_Server. These operations manage publications, one of the entities in the application domain. If a user requests the "show" operation for publication 1 through the corresponding endpoint, RS_Server returns a JSON response similar to the following example.

```json
{
  "id": 1,
  "doi": "10.1140/epjc/s10052-018-6047-y",
  "title": "Uncertainties in WIMP dark matter scattering revisited",
  "subject": null,
  "author": "John Ellis",
  "creator": "Springer",
  "producer": null,
  "pdf_url": "https://example.org/publication.pdf",
  "steadiness": 1.0,
  "score_rsm": 0.71,
  "score_trm": 0.68,
  "created_at": "2026-01-01T12:00:00.000Z",
  "updated_at": "2026-01-01T12:00:00.000Z",
  "url": "http://localhost:3000/publications/1.json"
}
```

3.3 Database

To support features such as storing link-annotated publications and authenticating users, RS_Server uses the database schema illustrated in Figure 3. The Readersourcing 2.0 application domain contains three main entities:

Endpoint	HTTP Message	Operation	Description
/publications.json	GET	Index	Fetches the entire collection of Publications.
/publications/list	GET	List	Fetches the entire collection of Publications (server-side view).
/publications/1.json	GET	Show	Returns the Publication with identifier equal to 1.
/publications/lookup.json	POST	Lookup	Searches for a Publication; if it does not exist, it is fetched from the given URL.
/publications/random.json	GET	Random	Returns a random Publication.
/publications/1/is_rated.json	GET	Is Rated	Checks if the Publication with identifier equal to 1 has been rated by at least one reader.
/publications/1/is_saved_for_later.json	GET	Is Saved For Later	Checks if the Publication with identifier equal to 1 has been saved for later by the current user.
/publications.json	POST	Create	Creates a new Publication.
/publications/is_fetchable.json	POST	Is Fetchable	Checks if the provided URL contains a fetchable Publication.
/publications/extract.json	POST	Extract	Extracts the rating URL from a link-annotated Publication.
/publications/fetch.json	POST	Fetch	Fetches a Publication from the given URL.
/publications/1/refresh.json	GET	Refresh	Fetches the Publication with identifier equal to 1 again.
/publications/1.json	PUT	Update	Updates the Publication with identifier equal to 1.
/publications/1.json	DELETE	Delete	Deletes the Publication with identifier equal to 1.
...	...	...	...

Table 1: Subset of the RESTful interface of RS_Server.

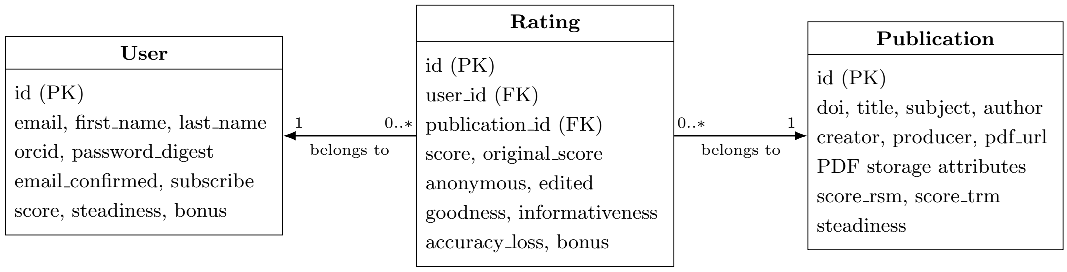


Figure 3: Persistent entities and associations in the RS_Server database (selected attributes).

- **Users**: represents the users of the system and stores their personal data. A user may have an optional *ORCID* and a boolean indicating whether they wish to receive emails. Additional attributes store the tokens required for operations such as password reset.
- **Ratings**: represents the ratings provided by readers for publications. Each rating has a numerical score, a boolean indicating whether it is anonymous, and another boolean recording whether it was edited later. A reader can provide at most one rating for a given publication; this invariant is enforced both by the application model and by a unique database constraint.
- **Publications**: represents publications known to the server, including those saved for later but not yet rated. A publication may have a *DOI*, descriptive metadata, and attributes used to manage the original and link-annotated PDF files.

These entities also contain attributes such as *steadiness* and *informativeness*, which store scores or parameters computed by the Readersourcing models.

Figure 3 depicts the two mandatory associations of a rating. A user may give zero or more ratings, but each rating belongs to exactly one user. An anonymous rating still retains this association so that authentication and the one-rating-per-user constraint can be enforced; anonymity controls how the reader identity participates in presentation and processing, not referential integrity.

Similarly, a publication may receive zero or more ratings, while each rating belongs to exactly one publication. The zero multiplicity is required because a publication may be fetched and saved for later before any rating is submitted.

3.4 Class Diagram

Figure 4 shows the main responsibilities of the RS_Server implementation. Rails controllers map HTTP operations to the domain models, while JSON and server-rendered views present their results. The three persistent models encapsulate the business logic and associations described in the previous section. Authentication and Readersourcing computations are delegated to focused command and strategy objects rather than being absorbed into the controllers.

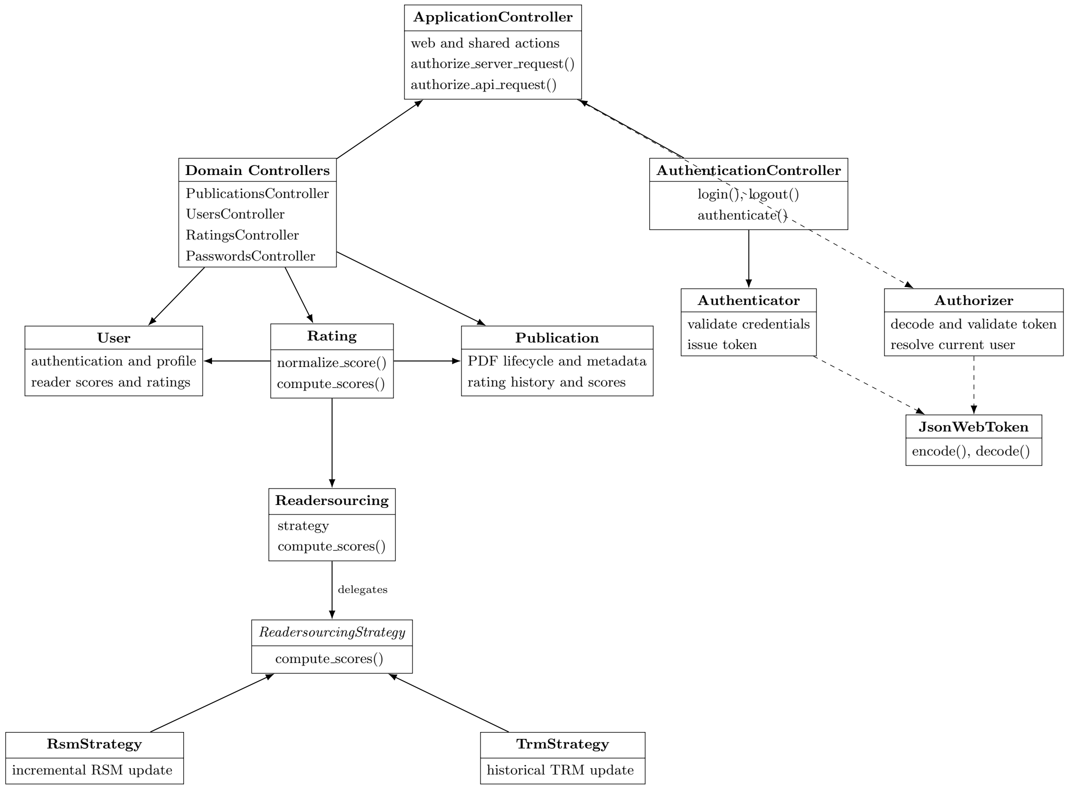


Figure 4: High-level class diagram of RS_Server. Method lists are intentionally selective.

The *ApplicationController* contains shared authorization hooks for both API and server-side requests, while the *AuthenticationController* handles login and logout. The *Authenticator* and *Authorizer* command objects isolate credential verification, token issuance, and token validation from HTTP concerns.

For REST clients, relying only on server-side session data to identify the logged-in user is unsuitable because the RESTful interface is *stateless*. Clients therefore attach an authentication *token* to each protected request. The server-rendered workflow can still keep the same token in its Rails session.

Authentication establishes the identity of the reader making the request, while resource-level authorization determines which records that reader may operate on. Profile updates, account deletion, and subscription changes are restricted to the current reader. A reader may inspect and edit only their own rating through the individual rating endpoints. Reader collections and individual reader responses expose the public academic profile—name, ORCID, and Readersourcing scores—without email or subscription preferences; the current reader obtains those private profile fields through the dedicated information endpoint.

When a user logs in, the supplied credentials are verified against the stored password digest and the account confirmation state. The credentials themselves are not placed in the token. If verification succeeds, RS_Server issues an HS256-signed JWT whose payload contains the user identifier, the client IP address, and a standard expiration claim set to seven days after issuance.

The browser client stores the returned token in its application cookie and attaches it to subsequent API requests through the `Authorization` header. The header accepts both the direct token format used by RS_Rate and the conventional `Bearer <token>` format. The server-rendered workflow also keeps a duplicate token in the Rails session, which is implemented through an HTTP-only cookie. Upon receiving a protected request, the *Authorizer* verifies the JWT signature, expiration time, IP address, and referenced user before allowing the request to proceed. Figure 5 provides an intuitive illustration of this authentication process.

The Rails session identifier is renewed after successful authentication, before the token is stored, and is renewed again when the reader logs out. Logout is a state-changing operation and is therefore available only through an HTTP `POST` request. HTML state-changing requests require a valid Rails authenticity token. JSON API requests remain stateless: a request carrying a JSON body, or an authenticated JSON request carrying an `Authorization` header, is handled through the token-based API boundary instead of the browser-session CSRF boundary.

Password recovery uses a separate, single-use credential. An HTTP `GET` request only displays the recovery form; sending the request requires `POST`. The request endpoint returns the same confirmation for registered and unregistered addresses. For a registered address, RS_Server generates a random token, stores only its SHA-256 digest and issue time, and sends a link built from the configured public origin. The link remains valid for four hours. It opens a server-rendered form in which the reader chooses and confirms a new password;

a successful reset clears the stored digest before a confirmation email is sent. Passwords are never sent by email.

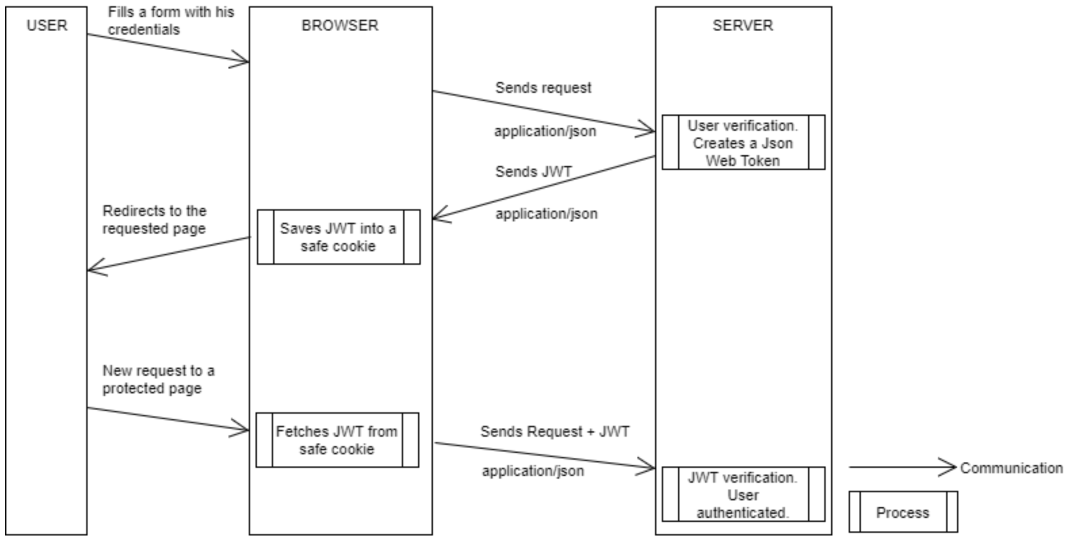


Figure 5: Representation of the token-based authentication process (NOT UML).

The classes responsible for the Readersourcing computations follow the *Strategy* pattern: *Readersourcing*, *ReadersourcingStrategy*, *RsmStrategy*, and *TrmStrategy*. A rating invokes the RSM strategy first and the TRM strategy second. The pattern keeps model selection extensible while expressing the score computation order explicitly.

3.5 Deploy

RS_Server supports two setup paths: a manual installation, intended primarily for development, and a Docker Compose deployment, which provides a reproducible application and database environment.

Environment-specific secrets and database settings are described in Section 3.5.3. A local `.env` file must never be committed.

3.5.1 Modality 1: Manual

This modality downloads and initializes RS_Server directly on the local machine and offers the greatest flexibility for development and domain-model experiments.

3.5.1.1 Requirements

- Ruby == 3.4.10;
- Bundler compatible with the committed `Gemfile.lock`;

- a Java 21 runtime, required by RS_PDF;
- PostgreSQL >= 17;
- Node.js >= 18, used to install and precompile the existing browser assets.

3.5.1.2 How To

Clone the RS_Server repository,[8] navigate to its root directory, create the `.env` file described in Section 3.5.4, and run:

```
bundle install
node .yarn/releases/yarn-3.6.3.cjs install --immutable
bin/rails db:create
bin/rails db:migrate
bin/rails db:seed  # optional sample data
bin/rails server -b 127.0.0.1 -p 3000 -e development
```

PostgreSQL must be accepting connections before the database commands are executed. With the sample bind address and port, the application is available at `http://127.0.0.1:3000`. Rails binstubs are always invoked from the repository root so that the locked application dependencies are used.

3.5.1.3 Quick Cheatsheet

1. change to the main directory with `cd`;
2. create and populate the `.env` file;
3. `bundle install`;
4. `node .yarn/releases/yarn-3.6.3.cjs install --immutable`;
5. `bin/rails db:create`;
6. `bin/rails db:migrate`;
7. `bin/rails db:seed` (optional);
8. `bin/rails server -b <address> -p <port> -e development`.

3.5.2 Modality 2: Docker Compose

Docker Compose builds RS_Server from the project source tree and starts it together with PostgreSQL 17. The application image includes Ruby 3.4.10, the Java 21 runtime required by RS_PDF, the locked Ruby dependencies, and the precompiled browser assets. This is the recommended reproducible deployment for local integration and production-like verification.

[8] `https://github.com/Miccighel/Readersourcing-2.0-RS_Server`

3.5.2.1 Requirements

- Docker Desktop or another installation supporting Docker Compose v2.[9]

3.5.2.2 How To

Clone the repository, create the `.env` file, ensure that the Docker Engine is running, and execute:

```
docker compose up --build
```

The `database` service must pass its health check before the `rs_server_webapp` service starts. The application entrypoint runs `bin/rails db:create` and `bin/rails db:migrate` automatically. Once startup completes, the web application is available at `http://localhost:3000`; `0.0.0.0` is the container bind address, not the address users should enter in a browser.

Sample data remain opt-in:

```
docker compose run --rm rs_server_webapp bin/rails db:seed
```

Use `docker compose down` to stop the services. The named PostgreSQL volume is retained so that application data survive ordinary container recreation.

Quick Cheatsheet

- change to the main directory with `cd`;
- create and populate the `.env` file;
- `docker compose up --build`;
- `docker compose run --rm rs_server_webapp bin/rails db:seed` (optional);
- `docker compose down` (to stop the services).

3.5.3 Environment Variables

Deployment-specific credentials and connection details are supplied through environment variables. Table 2 lists the variables used by the application. Database URLs take precedence over individual PostgreSQL settings.

Environment Variable	Purpose	Scope
`SECRET_DEV_KEY`	Rails secret used in development.	development

[9] `https://docs.docker.com/compose/`

Environment Variable	**Purpose**	**Scope**
`SECRET_PROD_KEY`	Rails secret used to sign and encrypt production data, including JWTs and session data.	production
`DATABASE_URL`	Complete PostgreSQL connection URL. Overrides the individual `POSTGRES_*` variables.	development, production
`TEST_DATABASE_URL`	Complete PostgreSQL connection URL for the test database.	test
`POSTGRES_USER`	PostgreSQL user name used when a full URL is not supplied.	all database environments
`POSTGRES_PASSWORD`	Password for the PostgreSQL user.	all database environments
`POSTGRES_HOST`	PostgreSQL host; defaults to `localhost` outside Compose and is set to `database` by Compose.	all database environments
`POSTGRES_DB`	Development or production database name; defaults to `rs_server`.	development, production
`POSTGRES_TEST_DB`	Test database name; defaults to `rs_server_test`.	test
`SMTP_USERNAME`, `SMTP_PASSWORD`	Credentials for the configured SMTP service.	production mail
`SMTP_DOMAIN_NAME`, `SMTP_DOMAIN_ADDRESS`	Sender domain and SMTP server address.	production mail
`EMAIL_BUG_REPORT`, `EMAIL_ADMIN`	Recipient addresses for reports and general contact messages.	development, production
`RAILS_LOG_TO_STDOUT`	When present, sends production Rails logs to standard output.	production
`RAILS_MAX_THREADS`	Maximum Active Record connection pool size; defaults to 5.	all environments
`PUBLIC_BASE_URL`	Public HTTP or HTTPS origin used to generate password-recovery links; required for recovery in production.	production
`CORS_ALLOWED_ORIGINS`	Comma-separated origins allowed to make cross-origin API requests. An omitted value disables CORS in production.	production

Environment Variable	Purpose	Scope
FORCE_SSL	Set to true when the public instance is served through HTTPS.	production
RS_PDF_MAX_DOWNLOAD_BYTES	Maximum accepted publication size in bytes; defaults to 52428800 (50 MiB).	development, production
RS_PDF_OPEN_TIMEOUT	Maximum connection-opening time in seconds; defaults to 5.	development, production
RS_PDF_READ_TIMEOUT	Maximum response-reading time in seconds; defaults to 20.	development, production
RS_PDF_PROCESS_TIMEOUT	Maximum RS_PDF execution time in seconds; defaults to 60.	development, production
RS_PDF_ALLOW_PRIVATE_NETWORKS	Set to true only when publications must be fetched from a trusted private network.	development, production

Table 2: Environment variables of RS_Server.

3.5.4 Setting Variables

For local and Compose deployments, create a `.env` file in the RS_Server root and populate it using `key=value` entries. Do not quote secrets unless the selected environment loader requires it, and never commit the file.

The following listing shows a valid `.env` file:

```
SECRET_PROD_KEY=replace-with-a-long-random-secret
POSTGRES_USER=postgres
POSTGRES_PASSWORD=replace-with-a-database-password
POSTGRES_DB=rs_server
SMTP_USERNAME=replace-with-an-smtp-user
SMTP_PASSWORD=replace-with-an-smtp-password
SMTP_DOMAIN_NAME=readersourcing.example
SMTP_DOMAIN_ADDRESS=smtp.example
EMAIL_BUG_REPORT=bugs@readersourcing.example
EMAIL_ADMIN=contact@readersourcing.example
PUBLIC_BASE_URL=https://readersourcing.example
CORS_ALLOWED_ORIGINS=https://client.readersourcing.example
FORCE_SSL=true
RAILS_LOG_TO_STDOUT=true
```

The value of `PUBLIC_BASE_URL` is an origin: it contains the scheme, host, and optional port, but no path, query string, fragment, or credentials. In production, a missing value makes password recovery unavailable rather than deriving a link from an untrusted request host.

CORS_ALLOWED_ORIGINS contains exact origins separated by commas. Supplying * deliberately opens the cross-origin policy and is inappropriate for an ordinary public deployment. RS_Rate requests browser permission for the selected server origin and therefore does not depend on a generated extension origin being listed here. Set FORCE_SSL=true only after HTTPS is available at the public origin.

Publication downloads reject private and reserved network addresses by default. The RS_PDF_ALLOW_PRIVATE_NETWORKS override is intended only for a controlled deployment in which trusted publications are deliberately served from such a network.

3.5.5 Sending Mails

RS_Server supports standard SMTP services for account confirmation, password recovery, password-change notifications, bug reports, and contact messages. Recovery messages contain the single-use link described above; the new password is entered only in RS_Server and is not included in either recovery or confirmation mail. The SMTP provider must expose a host, port 587 with STARTTLS, a user name, a password, and a verified sender domain. Provider-specific API keys can be placed in SMTP_PASSWORD when the provider uses an API key as its SMTP credential; the provider documentation defines the required values.

3.5.6 Connecting To The Database

A full connection string supplied through DATABASE_URL takes precedence over the individual POSTGRES_* variables in development and production. The test environment follows the same rule using TEST_DATABASE_URL and POSTGRES_TEST_DB.

```
ENV['DATABASE_URL'] ||
  "postgresql://#{ENV['POSTGRES_USER'] || 'postgres'}:" \
  "#{ENV['POSTGRES_PASSWORD']}@" \
  "#{ENV['POSTGRES_HOST'] || 'localhost'}/" \
  "#{ENV['POSTGRES_DB'] || 'rs_server'}"
```

3.5.7 Logging To The Standard Output

Development writes logs to standard output by default. In production, setting RAILS_LOG_TO_STDOUT configures a tagged logger on standard output, which is appropriate for Docker and other distributed deployments.

4 RS_PDF

RS_PDF [6] is the software library used by RS_Server to edit PDF files and add the required URL when a reader requests to save a publication for later reading. The current implementation appends a final page containing a clickable rating URL and a QR code, and stores the same URL in the PDF BaseUrl metadata field.

Its command-line interface allows RS_Server to invoke it directly. Because the components are deployed together, they do not require a separate network service or communication protocol. The executable is distributed as the self-contained `RS_PDF-v2.0.0-shaded.jar` and bundled in the RS_Server `lib` directory.

4.1 Implementation and Technology

RS_PDF is developed in Kotlin, an object-oriented programming language that interoperates with the Java Virtual Machine. This interoperability allows developers to reuse software distributed in JAR format and call Java classes directly from Kotlin. The current build targets Java 21 and uses Kotlin 2.4.

This programming language was chosen because it incorporates many modern features and receives robust support. More importantly, the underlying tool used to edit PDF files is *PDFBox*,[10] a Java library that provides a complete PDF manipulation toolkit. Therefore, RS_PDF serves as a wrapper for PDFBox and adds the required reference to the PDFs requested by readers. ZXing[11] generates the QR code placed beside the link annotation, Apache Commons CLI parses the execution parameters, and Log4j 2 provides logging.

Kotlin was created by JetBrains,[12] which announced first-class support for Android development with Google in 2017.[13] In the same year, JetBrains also announced Kotlin/Native, which compiles Kotlin programs to native binaries without requiring the JVM.

The Kotlin documentation includes an official comparison with Java,[14] and the language has been discussed extensively by software developers.[15]

4.2 Package Diagram

Figure 6 shows how RS_PDF is divided into packages and provides a high-level overview of its internal architecture.

Interaction with RS_Server begins in the **program** package. The *Program* object parses and validates the command-line options before delegating to the publication controller.

The **utils** package provides shared filesystem defaults, program constants, and logging configuration. Server addresses are supplied through the required URL argument and are not hard-coded as constants.

The **publications** package contains the PDF business logic. Its Controller/Model separation follows the same object-oriented intent as MVC without depending on Rails: the controller discovers one or more PDF files and creates the corresponding models, while each model loads a document, adds the link and QR code, writes metadata, and saves a new file. A View is unnecessary because the result is the edited PDF itself.

[10] `https://pdfbox.apache.org/`
[11] `https://github.com/zxing/zxing`
[12] `https://www.jetbrains.com/`
[13] `https://blog.jetbrains.com/kotlin/2017/05/kotlin-on-android-now-official/`
[14] `https://kotlinlang.org/docs/reference/comparison-to-java.html`
[15] `https://medium.com/@octskyward/why-kotlin-is-my-next-programming-language-c25c001e26e3`

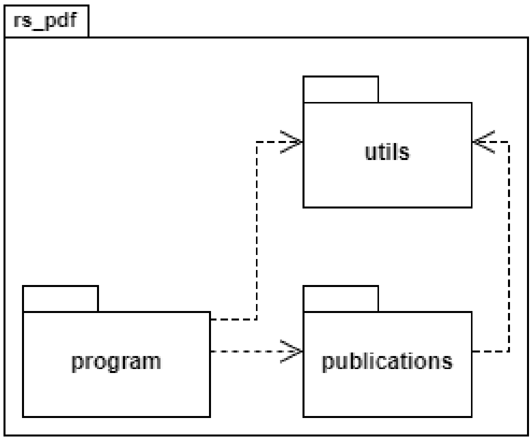


Figure 6: Package diagram of RS_PDF.

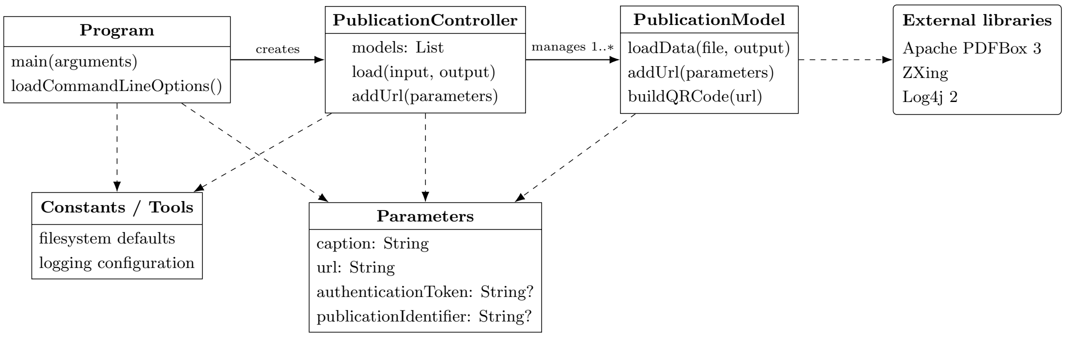


Figure 7: High-level class diagram of RS_PDF v2.0.0.

4.3 Class Diagram

Figure 7 shows the main classes of RS_PDF. The structure follows Controller/Model responsibilities: *Program* owns process-level parsing and flow, *PublicationController* manages the collection, and *PublicationModel* performs PDF transformations through PDFBox and ZXing.

The *Parameters* data class transports the caption, URL, optional authentication token, and optional publication identifier through the controller boundary. This keeps the *PublicationModel* interface stable as related input data evolve.

4.4 Installation

RS_PDF is bundled with compatible RS_Server distributions, eliminating the need for a separate installation. To use it independently, download the shaded JAR from the repository releases[16] or build it from source:

[16] `https://github.com/Miccighel/Readersourcing-2.0-RS_PDF`

```
./mvnw clean verify
```

On Windows use `mvnw.cmd clean verify`. The Maven Wrapper selects the configured Maven version and the executable is generated as `target/RS_PDF-v2.0.0-shaded.jar`.

4.4.1 Requirements

- Java runtime >= 21 to execute the shaded JAR;
- JDK >= 21 to compile and verify the project from source.

4.4.2 Command Line Interface

The behavior of RS_PDF is configured through the command-line options in Table 3. Caption and URL are always required. Input and output paths may be omitted only together, in which case the default `data` and `res` directories are used. The authentication token and publication identifier are optional, but if either is supplied then both custom paths and both server values must be supplied.

Short	**Long**	**Description**	**Constraint**
`-pIn`	`--pathIn`	Input PDF file or directory, using a relative or absolute path.	Optional; requires `-pOut`.
`-pOut`	`--pathOut`	Existing output directory for annotated PDF files.	Optional; requires `-pIn`.
`-c`	`--caption`	Caption of the clickable link added to the PDF.	Required.
`-u`	`--url`	Rating URL used by the link, QR code, and `BaseUrl` metadata.	Required valid URL.
`-a`	`--authToken`	Authentication token received from RS_Server.	Optional; requires paths and `-pId`.
`-pId`	`--publicationId`	Publication identifier received from RS_Server.	Optional; requires paths and `-a`.

Table 3: Command line options of RS_PDF.

The following example assumes that:

- there is a folder containing n PDF files at `C:\data`;
- annotated files must be saved in the existing folder `C:\out`;
- the shaded JAR is located at `C:\lib\RS_PDF-v2.0.0-shaded.jar`.

RS_PDF can then be executed with the following command:

```
java -jar C:\lib\RS_PDF-v2.0.0-shaded.jar \
  -pIn C:\data -pOut C:\out \
  -c "Express your rating" \
  -u "https://readersourcing.example/rate"
```

5 RS_Rate

RS_Rate [8] is an extension designed to function as a client for the Readersourcing 2.0 ecosystem without requiring access to its website. Compatible with Google Chrome,[17] Microsoft Edge,[18] and Mozilla Firefox,[19] the extension allows users to rate publications directly from their browsers. This eliminates the need to navigate to the main website, streamlining the process of providing ratings for publications.

The primary objective of RS_Rate is to enable readers to rate a publication with minimal effort: just a few clicks or keystrokes within the Readersourcing 2.0 ecosystem. In doing so, it contributes to a more dynamic online reading experience. RS_Rate is the first client developed for the project beyond the web-based interface available on the main portal.

Looking ahead, our vision includes expanding the compatibility of RS_Rate to Safari and other popular browsers. Our commitment is to make this rating tool accessible across a broad range of browsers, ensuring users can seamlessly interact with content and provide feedback regardless of their preferred browser. The current Chromium and Firefox targets share the same application source and rating logic.

5.1 Implementation and Technology

RS_Rate is an extension for Google Chrome, Microsoft Edge, and Mozilla Firefox. These extensions are developed using standard web technologies such as HTML, CSS, and JavaScript. They are collections of files packaged according to the requirements of each browser. The build process copies and checks all executable assets locally, then produces unpacked Manifest V3 extensions in `dist/chromium` and `dist/firefox`.

The JavaScript component also uses jQuery to simplify DOM selection and manipulation, event management, animations, and AJAX operations. RS_Rate relies on these AJAX features to provide responsive interactions with the server. Bootstrap 4 and its jQuery-based plugins provide the markup and interaction behavior used by the extension views. Dependencies remain declared in `package.json` and are resolved through the committed Yarn lockfile.

[17] https://www.google.com/chrome/
[18] https://www.microsoft.com/en-us/edge/
[19] https://www.mozilla.org/firefox/

5.2 Installation

RS_Rate is freely available on the Google Chrome Web Store. To use it with a Chromium-based browser, follow the link and install the currently available version from the store page:

- **Google Chrome and Microsoft Edge**: `https://chromewebstore.google.com/detail/readersourcing-20-rsrate/hlkdlngpijhdkbdlhmgeemffaoacjagg`

The repository also produces a Firefox Manifest V3 package ready for local validation and submission to Mozilla Add-ons. It uses the following stable Gecko identifier:

`rs-rate@readersourcing.org`

For local testing, open `about:debugging`, choose *This Firefox*, and load

`dist/firefox/manifest.json`

using *Load Temporary Add-on*. Distribution through Mozilla Add-ons requires validation and signing by Mozilla.

5.3 Development and Packaging

The development toolchain uses Node.js 24 LTS and Yarn 4.17.1. From the repository root:

```
corepack enable
yarn install --immutable
yarn verify
```

`yarn verify` builds both targets, checks that all runtime assets are local, validates the Firefox build with Mozilla's `web-ext`, and runs the client contract tests. The Firefox archive intended for Mozilla Add-ons is generated with:

```
yarn package:firefox
```

The ZIP is written to `artifacts`. The Firefox target requires Firefox 142 or later. Its manifest declares the account, browsing, website-interaction, and publication-content categories required by the Readersourcing workflow. The extension does not include an analytics channel.

Development builds initially target `http://localhost:3000`. Users can change the RS_Server host in the extension options, and the same configured host is used for normal API requests and PDF extraction. When the value is saved, the browser asks the reader to grant access to that HTTP or HTTPS origin. The permission covers only the selected origin, while any configured application path remains part of the server address.

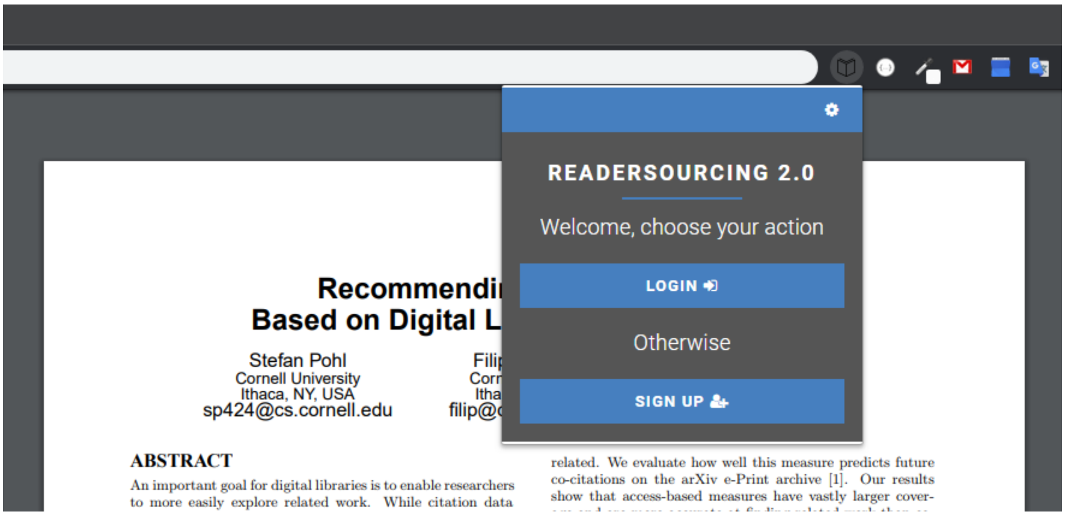


Figure 8: RS_Rate characterized as an extension having a popup action.

5.4 Usage

Figure 8 shows a Google Chrome window with the extension active for a publication. This represents a typical reader visiting a publisher's website to access a paper in PDF format. The figure also shows the first page displayed when the reader opens the client. From this page, the reader can reach either the login page shown in Figure 9 or the sign-up page. The login page also provides access to a password-recovery page, which requests the email containing the link to the RS_Server password form.

If a reader has yet to sign up for Readersourcing 2.0, they can navigate from the page shown in Figure 8 to the one displayed in Figure 11 and fill in the sign-up form. Once they complete the standard sign-up and login operations, they will find themselves on the rating page, as shown in Figure 10.

In the central section of the rating page, a reader can use the slider to choose a value in the 0–100 interval. After selecting the desired value, they submit the rating by clicking the green *Rate* button. The options page also allows the reader to make the rating anonymous. Readers must still be logged in when submitting an anonymous rating so that the system can prevent spam and repeated ratings of the same publication. Apart from these checks, the rating is processed without exposing the reader's identity.

If the reader prefers to rate the publication later, they can click the *Save for later* button. This option starts the publication-editing procedure, which stores a reference URL inside the PDF file being viewed. Once the procedure is complete, the *Save for later* button becomes a *Download* button, as shown in Figure 12.

The reader can then download the link-annotated publication. The refresh button, located to the right of the *Download* button, retrieves a new copy of the publication, annotates it, and makes it available again. This feature is useful when an author updates a publication

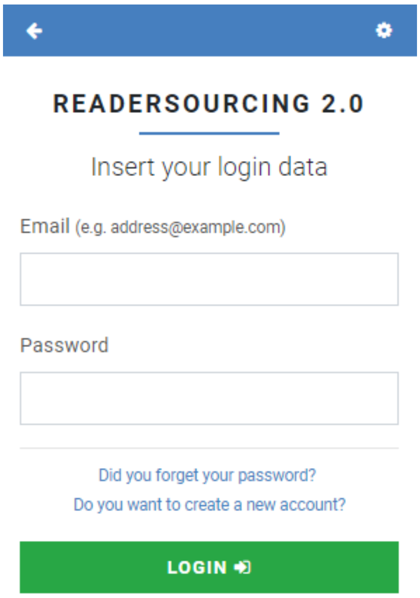


Figure 9: The login page of RS_Rate.

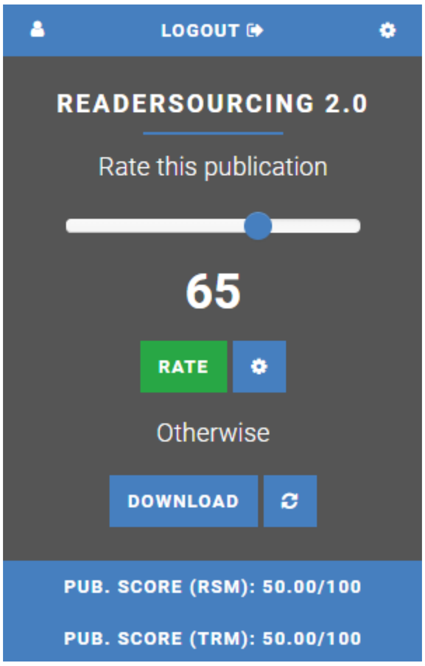


Figure 10: The rating page of RS_Rate.

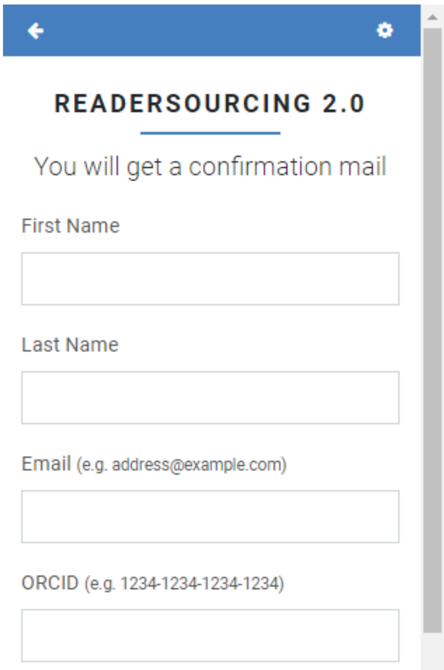


Figure 11: The user registration page of RS_Rate.

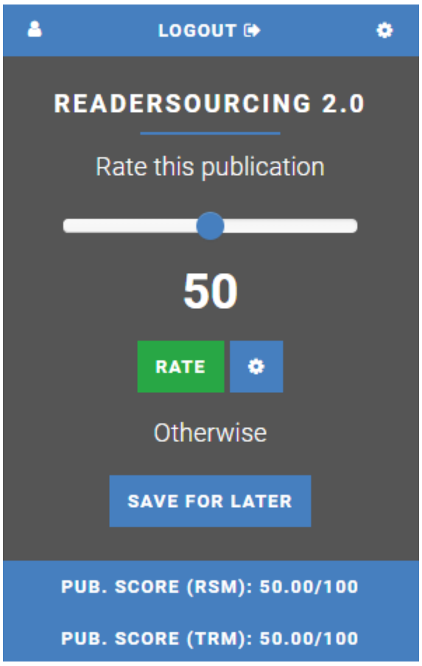
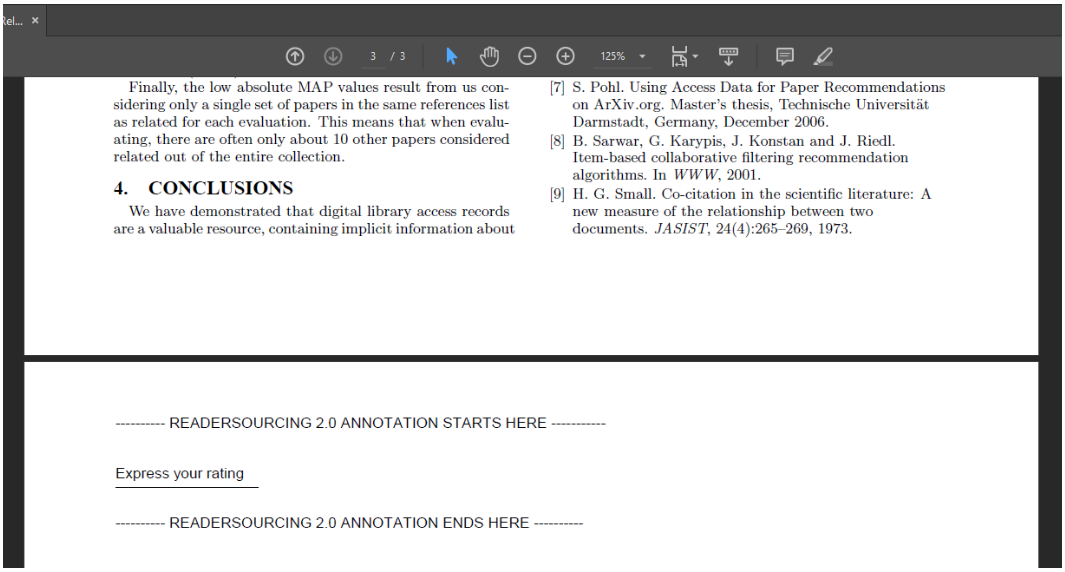


Figure 12: The rating page of RS_Rate after a "save for later" request.



Finally, the low absolute MAP values result from us considering only a single set of papers in the same references list as related for each evaluation. This means that when evaluating, there are often only about 10 other papers considered related out of the entire collection.

4. CONCLUSIONS

We have demonstrated that digital library access records are a valuable resource, containing implicit information about

[7] S. Pohl. Using Access Data for Paper Recommendations on ArXiv.org. Master's thesis, Technische Universität Darmstadt, Germany, December 2006.
[8] B. Sarwar, G. Karypis, J. Konstan and J. Riedl. Item-based collaborative filtering recommendation algorithms. In *WWW*, 2001.
[9] H. G. Small. Co-citation in the scientific literature: A new measure of the relationship between two documents. *JASIST*, 24(4):265–269, 1973.

---------- READERSOURCING 2.0 ANNOTATION STARTS HERE ----------

Express your rating

---------- READERSOURCING 2.0 ANNOTATION ENDS HERE ----------

Figure 13: A publication link-annotated through RS_Rate.

after the first download.

The downloaded PDF contains a new final page with the rating URL. Figure 13 shows an example opened in a PDF reader.

Following the reference takes the reader to a dedicated page in RS_Server. This interface allows them to rate the publication independently of the browser extension used to store the reference. For example, a reader can transfer the link-annotated publication to a tablet and use its built-in browser to submit the rating. Figure 14 shows the interface displayed after the reader follows the stored reference. The reader must authenticate again so that another person who receives a copy of the publication cannot use the reference on their behalf.

Whenever a reader rates a publication, RS_Server updates the scores computed by both the RSM and TRM models. RS_Rate displays the two publication scores at the bottom of the rating page, as shown in Figures 10 and 12. The reader can view their own scores by clicking the profile button in the upper-right corner. The profile page shown in Figure 15 also allows the reader to change their password.

6 RS_Py

RS_Py [7] is an additional component of the Readersourcing 2.0 ecosystem, providing a fully functional Python implementation of the models presented by Soprano et al. [5]. These models are also encapsulated by the server-side application of Readersourcing 2.0, as described in Section 3.

Developers familiar with Python can use RS_Py to generate and test new simulations of readers rating a set of publications. They can alter the internal logic of the models to test

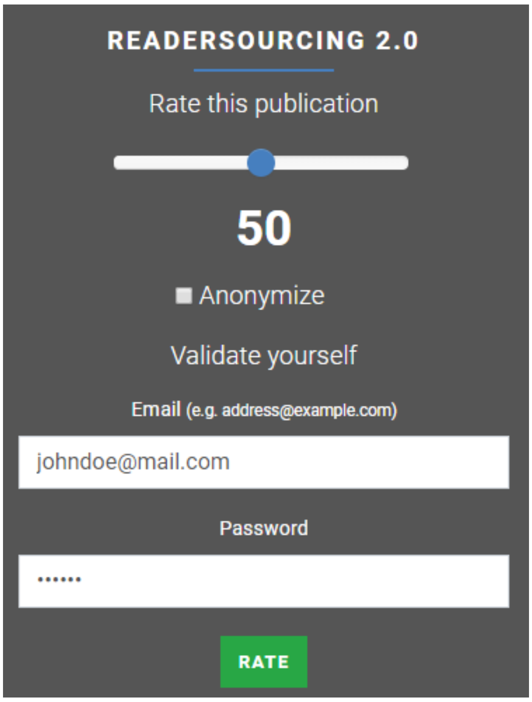


Figure 14: The server-side interface to rate a publication.

Figure 15: The profile page of RS_Rate.

new approaches without having to fork and edit the full Readersourcing 2.0 implementation.

6.1 Implementation and Technology

RS_Py is a collection of *Jupyter Notebooks*. Jupyter[20] is an open-source web application powered by Python[21] for creating and sharing documents containing live code, equations, visualizations, and narrative text. Notebooks are composed of cells that can be run independently, providing a step-by-step overview of the implemented computations.

6.2 Installation

To use the RS_Py notebooks, clone the repository[22] and place it somewhere on the filesystem. The project targets Python 3.13, and its tested direct dependencies are declared in `requirements.txt`. An isolated environment can be created as follows:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

On Windows, activate the environment with `.venv\Scripts\activate`.

6.3 Usage

RS_Py is organized into the following main folders:

- The `data` folder stores the datasets used to test the models presented by Soprano et al. [5] and the compressed synthetic datasets.
- The `models` folder stores the output of these models and the computed quantities.
- The `notebooks` folder contains Jupyter notebooks used to generate datasets, implement the models, and analyze experiments.
- The `scripts` folder is generated on demand and contains implementations of the Jupyter notebooks as pure Python scripts.
- The `src` folder contains the conversion utility and the in-memory domain implementation.
- The `tests` folder contains the numerical and utility characterization tests.
- The `utils` folder contains auxiliary tooling, including the power-law generator.

Within the `notebooks` folder, the principal files are:

[20] `https://jupyter.org/`
[21] `https://www.python.org/`
[22] `https://github.com/Miccighel/Readersourcing-2.0-RS_Py`

- `1.Seed-Power-Law.ipynb` generates synthetic data using power-law distributions;
- `2.Readersourcing.ipynb` implements and runs the RSM model;
- `3.TrueReview.ipynb` implements and runs the TRM model;
- `4.Experiments.ipynb` supports experimental analysis;
- `ReadersourcingToolkit.py` contains shared Python functionality used by the notebooks.

The `src/Convert.py` script converts the numbered notebooks into pure Python scripts and stores them in the generated `scripts` folder together with their shared toolkit.

The behavior of the notebooks can be customized by modifying the parameter settings found in their initial cells. To run them, navigate to the `notebooks` folder and type `jupyter lab`. This command starts the Jupyter server and opens the interface from which the notebooks can be executed in numerical order.

The in-memory implementation under `src/rs_py` and its characterization tests reproduce the reader, publication, and rating behavior currently shared with RS_Server. They complement rather than restrict the principal purpose of RS_Py: developers remain free to alter the internal logic of the notebooks and test new approaches. Additional object-oriented simulation work, including an explicit author entity, is maintained in the Readersourcing_OO repository.[23]

7 Overview

Figure 16 summarizes the capabilities of Readersourcing 2.0. In this example, four readers—`RD1`, `RD2`, `RD3`, and `RD4`—use RS_Rate to rate publication `P1`. Readers `RD1`, `RD2`, and `RD3` choose *Save for later* and receive a link-annotated version of the publication, denoted by `P1+Link`.

After some time, `RD1` opens `P1+Link` with a preferred PDF reader. `RD2` sends it to a tablet, while `RD3` opens it in a desktop browser. By following the link or scanning the QR code added through RS_PDF, they reach the dedicated RS_Server page and provide their ratings. In contrast, `RD4` rates `P1` immediately through the RS_Rate interface.

[23] `https://github.com/EddyMaddalena/Readersourcing_OO`

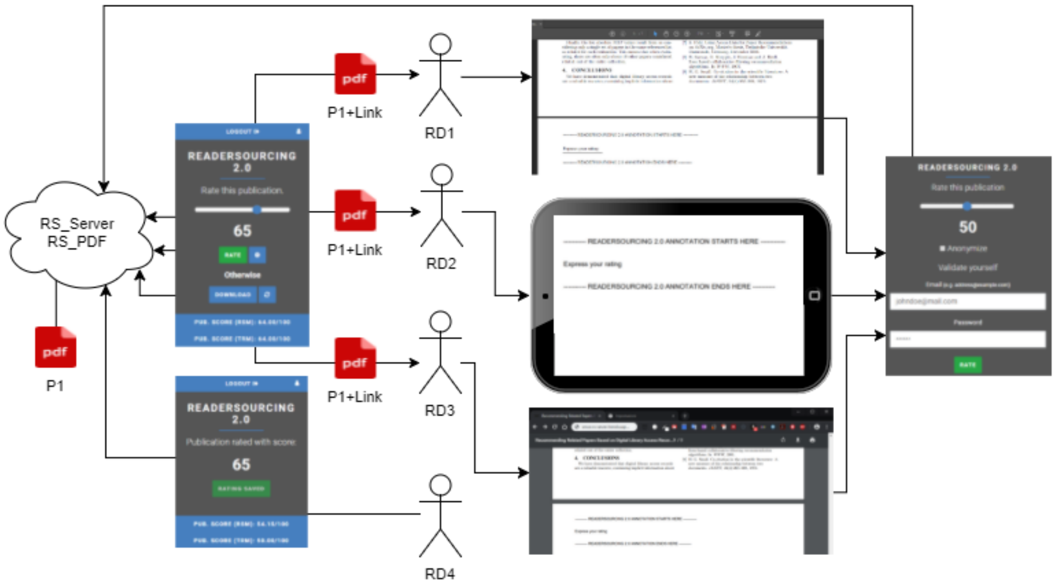


Figure 16: Readers' interaction modalities with the Readersourcing 2.0 ecosystem.

References

[1] Luca De Alfaro and Marco Faella. TrueReview: A Platform for Post-Publication Peer Review. In: *CoRR* abs/1608.07878 (2016). arXiv: `1608.07878`. URL: `http://arxiv.org/abs/1608.07878`.

[2] Martin Fowler. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd ed. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 2003. ISBN: 0321193687.

[3] Erich Gamma et al. *Design Patterns: Elementi per il Riuso di Software ad Oggetti*. Pearson, Jan. 2002.

[4] Stefano Mizzaro. Quality control in scholarly publishing: A new proposal. In: *Journal of the American Society for Information Science and Technology* 54.11 (2003), pp. 989–1005. DOI: `10.1002/asi.10296`. URL: `https://onlinelibrary.wiley.com/doi/abs/10.1002/asi.10296`.

[5] Michael Soprano and Stefano Mizzaro. Crowdsourcing Peer Review: As We May Do. In: *Digital Libraries: Supporting Open Science*. Vol. 988. Communications in Computer and Information Science. Springer, 2019, pp. 259–273. DOI: `10.1007/978-3-030-11226-4_21`. URL: `https://doi.org/10.1007/978-3-030-11226-4_21`.

[6] Michael Soprano and Stefano Mizzaro. *Readersourcing 2.0: RS_PDF*. DOI: `10.5281/zenodo.1442597`. URL: `https://doi.org/10.5281/zenodo.1442597`.

[7] Michael Soprano and Stefano Mizzaro. *Readersourcing 2.0: RS_Py*. DOI: 10.5281/zenodo.3245208. URL: https://doi.org/10.5281/zenodo.3245208.

[8] Michael Soprano and Stefano Mizzaro. *Readersourcing 2.0: RS_Rate*. DOI: 10.5281/zenodo.1442599. URL: https://doi.org/10.5281/zenodo.1442599.

[9] Michael Soprano and Stefano Mizzaro. *Readersourcing 2.0: RS_Server*. DOI: 10.5281/zenodo.1442630. URL: https://doi.org/10.5281/zenodo.1442630.

[10] Michael Soprano, Kevin Roitero, and Stefano Mizzaro. HITS Hits Readersourcing: Validating Peer Review Alternatives Using Network Analysis. In: *Proceedings of the 4th Joint Workshop on Bibliometric-enhanced Information Retrieval and Natural Language Processing for Digital Libraries*. Vol. 2414. CEUR Workshop Proceedings. 2019, pp. 70–82. URL: https://ceur-ws.org/Vol-2414/paper7.pdf.

[11] Michael Soprano et al. Evaluation of Crowdsourced Peer Review using Synthetic Data and Simulations. In: *Proceedings of the 21st Conference on Information and Research Science Connecting to Digital and Library Science*. Vol. 3937. CEUR Workshop Proceedings. 2025. URL: https://ceur-ws.org/Vol-3937/paper8.pdf.